

Abstract

Contents

1	Introduction	1
1.1	Research goal	1
2	Background	2
2.1	Common Workflow Language	2
2.2	ASTRON and DAS-6 and Low Frequency Array (LOFAR)	3
2.3	Dask	3
3	Related work	4
3.1	Workflow Runners	4
3.2	Python Workflow Definition	4
3.3	Parsl	4

1 Introduction

In the age of big data, processing large amounts of data efficiently is a constant challenge. Organizations in scientific, industrial, and commercial domains must continuously innovate to handle the ever-growing volume and complexity of data.

An example of modern data processing is the Low Frequency Array (LOFAR) telescope operated by the Netherlands Institute for Radio Astronomy (ASTRON) [kd: cite], which has measuring stations throughout Europe. By combining the enormous amounts of measurement data produced by each station, LOFAR acts as the largest radio telescope operating at the lowest frequencies that can be observed from Earth. The merging and further processing of the telescope measurements is done by a variety of data processing pipelines that require large amounts of compute power to complete in a reasonable time frame. ASTRON runs the pipelines on the DAS-6 supercomputer [1], a modern distributed system shared between a group of scientific institutes in the Netherlands. To automate data processing pipelines, such as the used for LOFAR, pipeline tools or steps are chained into workflows. There are many options for creating and running workflows. As of writing, there exist at least 366 workflow systems [2]. Specifically, ASTRON uses Common Workflow Language (CWL) [3] to create workflows. CWL is a standard for describing tools and workflows with YAML files and was created to increase workflow portability and reusability between different research groups and workflow systems. Because CWL is purely a descriptive method for defining workflows, a second system is needed to parse and execute the workflow. ASTRON uses Toil [4] as an open-source workflow engine to run their pipelines on DAS-6.

Where huge amounts of data and complex processing pipelines meet, small optimizations can already significantly reduce the required compute resources and/or time. However, CWL inherently lacks methods to optimize workflows and Toil only optimizes workflows by running independent tools in parallel. Feeling that further performance gains can be obtained from their pipelines, ASTRON has started looking for more flexible solutions. ASTRON has indicated a specific interest in exploring workflow execution with Dask [5], a Python parallel programming library. However, while using Python to construct tools and workflows should enable problem-specific optimization options, the advantages of being part of the CWL ecosystem would disappear. In addition, replacing all CWL tools and workflows with Dask would cost resources: It is necessary to be able to automatically transform CWL tools and workflows to Dask workflows. Lastly, pivoting away from CWL to Python-based solution should not require workflow developers to learn a new system with a steep learning curve. A new solution should be usable for developers with basic Python skills.

1.1 Research goal

The main objective of this work is to automatically transpile CWL processes into easily adjustable and extendable Python scripts that developers can further improve upon, merging workflow description and execution into a single Dask-based workflow system. Furthermore, the new system must be able to execute workflows on distributed systems, such as the DAS-6 supercomputer. Lastly, code abstractions need to be determined that hide more complex code from basic users and that reduce visual clutter caused by repeated code blocks to prevent users from feeling overwhelmed.

This work describes the development process of 1.) An in-Python format that describes workflows and uses Dask as workflow runner that mimics a CWL workflow; and 2.) A transpiler that transforms

CWL workflows into a developed in-Python format. To keep the project scope in check, this work focuses on a proof-of-concept that ports a subset of the CWL standard feature set. Finally, the solutions are developed by answering the following questions.

Q1: How can Dask be used to run tasks on a distributed system?

Dask is an extensive library that offers task parallelism on multiple abstraction levels. Exploring the use-cases of each option is crucial for deciding how transpiled CWL workflows should fundamentally function. Moreover, exploring and reporting on any configurational necessities for executing Dask on various platforms should help reduce the learning curve for workflow developers.

Q2: How can CWL workflows be transformed into tasks that can be executed by Dask?

After settling on how Dask should be used for task execution, an in-Python-written format needs to be developed that mimics CWL workflow description and execution. Simplicity is key and exploring what CWL functionality should be abstracted away to reduce overall complexity will improve usability. A specific CWL feature that needs considering are CWL expressions. YAML lacks inherent scripting support, which the CWL standard copes with by allowing Javascript expressions in CWL files that are evaluated at runtime by the workflow runner. CWL expressions may reference special objects available at runtime. It is important to explore how a Dask solution should handle these expressions.



2 Background

2.1 Common Workflow Language

The Common Workflow Language (CWL) project [3] was founded to create a set of open standards specifying how to describe tools, workflows, their inputs and outputs, and how workflow runners should execute workflows. Modern data processing tasks may consist of many scripts or programs that read data, run computations, and pass on new data to other tools. At first, automatic execution of workflows was handled by ad-hoc scripts. These scripts were often platform specific and required adjusting when used on other systems, when integrated with other workflows, or when shared with outsiders. Different parties quickly developed ways to describe and run workflows and their tools independently of the underlying platform, but a general standard was still missing. CWL was an answer to the problematic reproducibility and interoperability between research teams. Tools and workflows were rewritten in CWL, making the sharing and reuse of tools and workflows almost trivial. Many workflow runners, such as Toil and Arvados [4, 6], were updated to support CWL workflows.

The CWL standard defines tools and workflows as executable processes written in YAML [7]. Processes are supposed to be non-interactive: A process is given inputs, followed by the execution of the process and its subprocesses, and terminates after producing the declared outputs. CommandLineTools are the basic building blocks of a workflow that describe how to transform inputs into an executable shell command. CWL supports containerized tool execution with Docker and Singularity [8, 9]. Workflows describe data dependencies between processes in so called steps. Each step runs a CWL process, which can be both a tool or workflow. Workflow steps declare

which data should be passed to its underlying process and what outputs that underlying process produces as a reference for other steps. The input data is either sourced from the process inputs or the outputs of other steps. The CWL standard specifies many methods that alter tool and workflow behaviour, like conditional step execution, scatter-gather operations, and the possibility to write Javascript expressions that are evaluated at runtime. Expressions may reference certain environment-dependent variables and introduce dynamic behaviour to otherwise static tool and workflow definitions.

Tools and workflows are executed by workflow runners. A workflow runner is responsible for validating CWL process definitions, scheduling and executing CommandLineTools that are ready for execution, managing input and output data, and managing execution failures. The CWL project provides a reference workflow runner, called cwltool, that is implemented in Python. In addition, another Python library called cwl-utils offers methods to work with CWL documents.



2.2 ASTRON and DAS-6 and Low Frequency Array (LOFAR)

[kd: *Depends on experiments*]

2.3 Dask

Dask is a Python library designed to trivially parallelize existing Python code. Python is often used by the scientific community for its quick prototyping capabilities and, more recently, to act as a high level driver for data science and machine learning [10]. Much of the computational code is written using Pandas [11] and NumPy [12]. Both libraries allow for efficient manipulation of large volumes of data with a fairly simple API. However, scaling these data manipulations beyond single core execution is non-trivial, as the libraries were initially not built for this purpose.

Dask [5] was developed as a solution that enables scaling of scientific code by mimicking the APIs of standard Python, Pandas, and NumPy on a high level, taking care of scaling in the background, limiting the need for large rewrites. Dask achieves a platform independent API by using a platform dependent cluster object that offers a client for programs to interact with: Developers only need to change the cluster object when switching between platforms. Examples of available cluster objects include one for local deployment (local cluster), for deployment via a major cloud provider (AWS, GCP, Azure) [kd: [?]], for HPC systems via resource managers like SLURM [kd: [?]], and for kubernetes clusters [kd: [?]].

Dask offers both data-oriented and task-oriented parallelism. Parallelizing data heavy computations is achieved by loading data into Pandas or NumPy datastructures. Dask can automatically divide computations on these large datastructures into multiple smaller datasets that are split between computational units, whether they reside on a single computer or across multiple computing instances within a cluster. In addition to data-oriented parallelism, Dask offers a scalable version of the `concurrency.futures` API from the Python standard library. Tasks can be scheduled for execution by submitting a task graph of `Delayed` functions, or by submitting functions to the Dask client. The intuitiveness and flexibility of Dask have made it a commonly used solution for parallelize Python code.

3 Related work

3.1 Workflow Runners

3.2 Python Workflow Definition

3.3 Parsl

Parisl [13] is a Python library for running workflows in parallel on a variety of platforms. A tool is created by wrapping a Python function in a `python_app` or, in the case of shell commands, a `bash_app`. Similarly to Dask `Delayed` objects, apps return futures representing outputs that can be passed as input to other apps, implicitly creating a workflow in the form of a dependency graph. Parsl has deployment support for many distributed systems

Attempting to integrate the CWL ecosystem into Parsl, the `cwl-parisl` library was created [14]. In addition to `python_apps` and `bash_apps`, `CWLApps` are introduced that read CWL `CommandLineTools` from YAML files and internally wrap them in a `bash_app`.

In an attempt to remove the need for Javascript in a Python runtime environment, the CWL+Parisl work introduces the Python expression as a proof of concept. Similarly to CWL Javascript expressions, Python expressions store code as strings and can reference special objects. Our transpiler goes a step further by removing the need for special expressions altogether, although Javascript expressions are still supported for CWL compatibility.

While `CWLApps` can be used to integrate CWL `CommandLineTools` in Parsl workflows, the library lacks a method to integrate CWL workflows directly into Parsl, limiting interoperability to just `CommandLineTools`. Additionally, the inner workings of `CWLApps` cannot be easily adjusted without overriding the class, something this work tries to accomplish.

References

- [1] Henri Bal, Dick Epema, Cees De Laat, Rob Van Nieuwpoort, John Romein, Frank Seinstra, Cees Snoek, and Harry Wijshoff. A medium-scale distributed system for computer science research: Infrastructure for the long term. *Computer*, 49(5):54–63, 2016.
- [2] Michael R. Crusoe Nebojša Tijanić Samuel Lampa et al. Peter Amstutz, Maxim Mikheev. Existing workflow systems. Common Workflow Language wiki, GitHub: <https://s.apache.org/existing-workflow-systems>, 2026. Accessed 08-09-2026.
- [3] Michael R Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, et al. Methods included: standardizing computational reuse and portability with the common workflow language. *Communications of the ACM*, 65(6):54–63, 2022.
- [4] John Vivian, Arjun Arkal Rao, Frank Austin Nothaft, Christopher Ketchum, Joel Armstrong, Adam Novak, Jacob Pfeil, Jake Narkizian, Alden D Deran, Audrey Musselman-Brown, et al. Toil enables reproducible, open source, big biomedical data analyses. *Nature biotechnology*, 35(4):314–316, 2017.

- [5] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *SciPy*, 2015.
- [6] Arvados authors. Arvados — Documentation — doc.arvados.org. <https://doc.arvados.org/>. [Accessed 06-01-2026].
- [7] Oren Ben-Kiki, Clark Evans, and Brian Ingerson. Yaml ain’t markup language (yaml™) version 1.2. <https://www.earthdata.nasa.gov/s3fs-public/imported/YAML%201.2%20Spec.pdf>, 2009.
- [8] Inc Docker et al. Docker. *linea*].[Junio de 2017]. Disponible en: <https://www.docker.com/what-docker>, 2020.
- [9] Gregory M Kurtzer, Vanessa Sochat, and Michael W Bauer. Singularity: Scientific containers for mobility of compute. *PloS one*, 12(5):e0177459, 2017.
- [10] Travis E Oliphant. Python for scientific computing. *Computing in science & engineering*, 9(3):10–20, 2007.
- [11] Wes McKinney et al. pandas: a foundational python library for data analysis and statistics. *Python for high performance and scientific computing*, 14(9):1–9, 2011.
- [12] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *nature*, 585(7825):357–362, 2020.
- [13] Yadu Babuji, Anna Woodard, Zhuozhao Li, Daniel S Katz, Ben Clifford, Rohan Kumar, Lukasz Lacinski, Ryan Chard, Justin M Wozniak, Ian Foster, et al. Parsl: Pervasive parallel programming in python. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, pages 25–36, 2019.
- [14] Nishchay Karle, Ben Clifford, Yadu Babuji, Ryan Chard, Daniel S Katz, and Kyle Chard. Parsl+ cwl: Towards combining the python and cwl ecosystems. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 2040–2047. IEEE, 2024.